FASE 3 · ATIVIDADE INTEGRADORA

FIAP · Ciência da Computação

RELATÓRIO TÉCNICO · FASE 3

Aurora Siger

Sistema inteligente de análise e decisão da colônia

ALUNO **Gabriel Tancler Armada**

RM 574883

TURMA 1CCOR-2026

DATA Novembro de 2026

RESUMO

O sistema lê os dados da colônia Aurora Siger, prevê a geração eólica da próxima hora e decide o que fazer com a energia. Os dados ficam num dicionário hierárquico (energia, ambiente e operação) e o histórico de leituras, em listas. Uma regressão linear simples, calculada à mão pelo método dos mínimos quadrados, estima a geração eólica a partir do vento (R^2 = 0,9987). Regras com `if` / `elif` / `else` combinam nível da bateria, consumo e previsão para gerar alertas, ativar o modo economia e desligar módulos não essenciais na ordem de prioridade, sem nunca desligar o suporte à vida. Na operação simulada, o consumo cai de 70 kW para 48 kW e o excedente de 4,3 kWh vai para a bateria em vez de ser perdido.

1 Visão geral

O sistema segue o fluxo entrada → processamento → saída pedido no enunciado. A entrada são os dados atuais da colônia e o histórico de leituras; o processamento é a previsão e a aplicação das regras; a saída são mensagens objetivas no formato "ALERTA: ..." ou "SUGESTÃO: ...". Cada etapa fica num arquivo próprio, o que separa dados, modelo e decisões (Tabela 1). O código usa apenas Python puro (3.8 ou superior), sem bibliotecas externas, e é organizado em funções com parâmetros e retorno, como no Cap. 3.

Tabela 1: Arquivos do sistema

Arquivo	Papel	Funções principais
`dados.py`	Estado da colônia e histórico (entrada)	`ler`, `gravar`, `listar`, `mostrar_hierarquia`
`previsao.py`	Regressão linear vento → geração eólica	`ajustar_reta`, `prever`, `coeficiente_r2`
`decisoes.py`	Regras, priorização e análise de energia (saída)	`avaliar_regras`, `priorizar`, `analisar_energia`
`main.py`	Junta as partes e mostra a operação e os exemplos	`operacao_atual`, `exemplos_do_enunciado`

2 Organização dos dados

O estado atual da colônia fica num dicionário cujo valor, em cada chave, pode ser outro dicionário. Isso junta as duas estruturas do Cap. 4. O dicionário é uma tabela hash: a chave leva direto ao valor, com acesso médio O(1), sem percorrer os outros itens, o que atende ao "acesso rápido a informações críticas" do enunciado. O aninhamento forma uma árvore, com raiz, nós e folhas, do mesmo jeito que a hierarquia de arquivos de um sistema operacional citada no capítulo. O exemplo do enunciado (sistema energético → solar / eólico) aparece literalmente no ramo `energia`:

```
colonia
├── energia
│   ├── solar ......... geracao_kw = 25
│   ├── eolica ........ geracao_kw = 27 (última leitura)
│   └── bateria ....... capacidade_kwh = 200, carga_kwh = 80
├── ambiente .......... temperatura_interna_c, temperatura_externa_c, vento_ms = 11
└── operacional
    └── modulos ....... 7 módulos, cada um com consumo_kw, prioridade, essencial e ligado
```

A navegação entre subsistemas usa um caminho de texto: `ler("energia/bateria/carga_kwh")` desce pela árvore chave por chave e devolve 80; um caminho inexistente devolve `None` em vez de travar o programa. `gravar` troca um valor que já existe, `listar` mostra o que há dentro de um nó e `mostrar_hierarquia` imprime a árvore inteira chamando a si mesma a cada nível, que é a recursividade do Cap. 3.

O histórico fica fora do dicionário, em duas listas paralelas (`vento_ms` e `eolica_kw`), em que a posição *i* das duas é a mesma leitura. Lista é a estrutura adequada porque a ordem das leituras importa e a regressão percorre os pares em sequência; é também o formato do exemplo do enunciado (vento = [8, 10, 12], energia = [20, 25, 30]). Assim, `colonia` representa o agora e `historico`, o passado.

Os valores foram escolhidos para reproduzir o exemplo do enunciado: a bateria tem 80 de 200 kWh, ou seja, 40 % ("energia = 40"), e os módulos somam 70 kW ("consumo = 70"). Os módulos estão na Tabela 2; a prioridade 1 é a mais importante.

Tabela 2: Módulos da colônia

Módulo	Consumo (kW)	Prioridade	Essencial
Suporte à vida	20	1	sim
Habitação	12	2	sim
Estufa	10	3	sim
Comunicação	6	4	sim
Laboratório	9	5	não
Oficina	8	6	não
Iluminação externa	5	7	não
Total	**70**		

3 Regras de decisão

3.1 Regras de energia

As regras usam três condições, com os limites da Tabela 3. Cada uma vira uma variável booleana no código, como nas expressões do Cap. 2.

Tabela 3: Condições das regras

Variável	Condição	Significado
B (`energia_baixa`)	carga da bateria < 50 %	reserva baixa
A (`consumo_alto`)	consumo > 60 kW	módulos puxando muita energia
P (`previsao_ruim`)	eólica prevista < 20 kW	pouco vento na próxima hora (abaixo de cerca de 8 m/s)

A cadeia `if` / `elif` / `else` testa da situação mais grave para a mais leve, então só um bloco é executado. Escrita como expressões booleanas, a ordem dos testes equivale a:

$$CRÍTICO = B \cdot A \cdot P \quad \text{modo economia e desligar não essenciais} \tag{1}$$

$$ECONOMIA = B \cdot A \cdot \neg P \quad \text{reduzir consumo e ativar modo economia} \tag{2}$$

$$REDUZIR = B \cdot \neg A \quad \text{reduzir consumo} \tag{3}$$

$$NORMAL = \neg B \quad \text{nenhuma ação} \tag{4}$$

As quatro saídas são mutuamente exclusivas e cobrem as oito combinações de B, A e P. A regra básica do enunciado ("se energia < 50 → reduzir consumo") é a (3), e a combinada ("energia < 50 E consumo alto → modo economia") é a (2). Com a entrada do enunciado, energia = 40 e consumo = 70, a saída inclui "ALERTA: reduzir consumo" e também ativa o modo economia, porque 70 kW passa do limite de consumo alto. O trecho do código:

```
if energia_baixa and consumo_alto and previsao_ruim:
    return ["ALERTA CRÍTICO: energia baixa, consumo alto e pouco vento previsto",
            "AÇÃO: ativar modo economia e desligar sistemas não essenciais"]
elif energia_baixa and consumo_alto:
    return ["ALERTA: reduzir consumo",
            "AÇÃO: ativar modo economia"]
elif energia_baixa:
    return ["ALERTA: reduzir consumo"]
else:
    return ["NORMAL: energia suficiente, nenhuma ação necessária"]
```

3.2 Priorização

Quando o consumo dos módulos ligados passa da geração prevista, `priorizar` procura, entre os módulos não essenciais ainda ligados, o de maior número de prioridade (o menos importante), desliga esse módulo e repete até o consumo caber na geração. Módulos essenciais nunca entram na busca, então o suporte à vida, a habitação, a estufa e a comunicação continuam ligados mesmo com geração zero. Se só restarem essenciais e ainda faltar energia, a diferença sai da bateria.

3.3 Análise de energia

`analisar_energia` faz o balanço de uma hora (kW × 1 h = kWh) entre geração, consumo e bateria:

- consumo > geração: "ALERTA: consumo maior que geração", e a bateria cobre a falta; se a carga não for suficiente, "ALERTA CRÍTICO" com a quantidade que falta;
- geração > consumo: "SUGESTÃO: armazenar energia excedente" até a capacidade da bateria; o que não couber gera um aviso de desperdício;
- geração igual ao consumo: situação normal.

4 Modelo de previsão

A previsão usa a regressão linear simples do Cap. 7, com x = velocidade do vento (m/s) e y = geração eólica (kW). Os coeficientes saem do método dos mínimos quadrados, que escolhe a reta com a menor soma dos quadrados dos resíduos:

$$b_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma(x_i - \bar{x})^2, \quad b_0 = \bar{y} - b_1 \cdot \bar{x} \tag{5}$$

As fórmulas foram escritas à mão em `ajustar_reta`, sem bibliotecas. Com as oito leituras do histórico, $\bar{x}$ = 9,875 m/s e $\bar{y}$ = 24,5 kW; os somatórios são $\Sigma(x_i - \bar{x})(y_i - \bar{y})$ = 147,5 e $\Sigma(x_i - \bar{x})^2$ = 58,875, o que dá b_1 = 2,505 e b_0 = −0,240:

$$\hat{y} = -0{,}240 + 2{,}505 \cdot x \qquad \text{geração eólica prevista (kW)} \tag{6}$$

A inclinação indica que cada 1 m/s a mais de vento acrescenta cerca de 2,5 kW de geração. O R^2, também do Cap. 7, mede quanto da variação da geração a reta explica:

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 \,/\, \Sigma(y_i - \bar{y})^2 = 1 - 0{,}467 \,/\, 370 = 0{,}9987 \tag{7}$$

Para vento = 11 m/s, a reta prevê 27,3 kW, o "≈ 27" do enunciado (Figura 1). Os resíduos ficam entre −0,3 e 0,2 kW. O programa recusa listas de tamanhos diferentes ou com todos os valores de vento iguais, caso em que a fórmula de b_1 dividiria por zero.

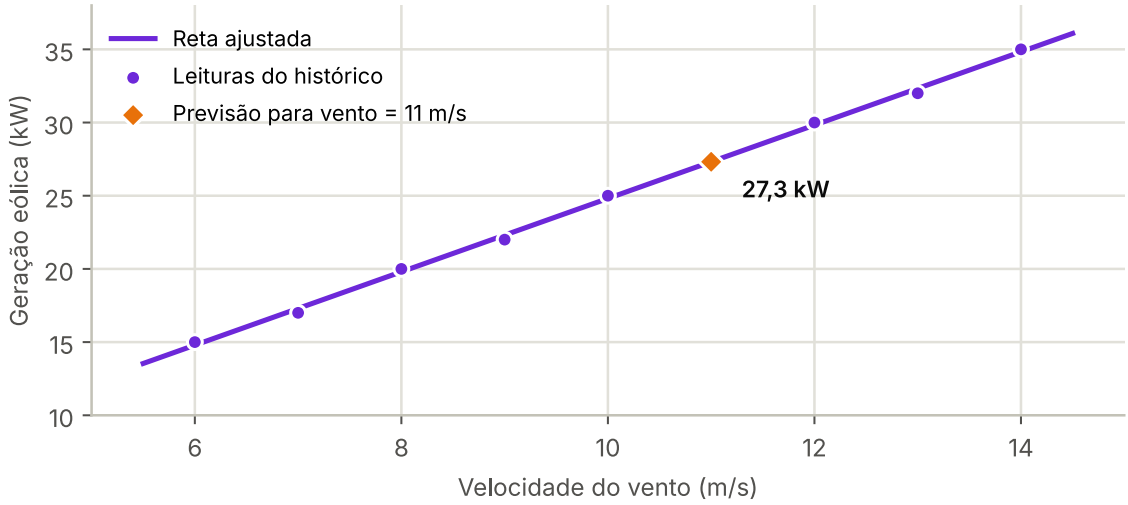


Figura 1: Leituras do histórico, reta ajustada e previsão para vento = 11 m/s.

Limitação. Pelo Cap. 8, a potência contida no vento é $P = \frac{1}{2} \cdot \rho \cdot A \cdot v^3$, ou seja, cresce com o cubo da velocidade. De 11 para 14 m/s, a potência teórica dobra (×2,06), enquanto a reta prevê aumento de 28 %. A reta descreve bem a turbina dentro da faixa observada (6 a 14 m/s), mas não deve ser usada fora dela. A curva de operação do mesmo capítulo também tem limites que uma reta não representa: abaixo da velocidade de partida (2 a 4 m/s) a turbina não gera, na velocidade nominal (cerca de 10 a 15 m/s) a potência para de subir, e acima da velocidade de corte (cerca de 20 a 25 m/s) a turbina é desligada por segurança.

5 Como o sistema melhora o uso de energia

A Tabela 4 resume a operação da próxima hora com os dados atuais da colônia, como o programa imprime.

Tabela 4: Operação simulada da próxima hora

Etapa	Resultado
Entrada	bateria 80/200 kWh (40 %), consumo 70 kW, vento 11 m/s
Previsão	eólica 27,3 kW + solar 25 kW = geração de 52,3 kW
Regras	"ALERTA: reduzir consumo" e "AÇÃO: ativar modo economia" (regra 2)
Priorização	desliga iluminação externa (5 kW), oficina (8 kW) e laboratório (9 kW); consumo cai para 48 kW
Análise de energia	"SUGESTÃO: armazenar energia excedente": 4,3 kWh vão para a bateria, que termina com 84,3 kWh

O ganho vem de três pontos. Primeiro, o sistema decide pelo que vai acontecer, não só pelo que já aconteceu: usar a previsão para a próxima hora em vez da última leitura é o que o enunciado chama de passar de um sistema reativo para um preditivo, e a regra crítica (1) só existe porque há previsão. Segundo, o corte de carga é seletivo e para assim que o consumo cabe na geração: sem a priorização, a colônia teria um déficit de 17,7 kW tirado da bateria, que já está abaixo da metade; com ela, a bateria passa a ganhar carga e nenhum sistema vital é afetado. Terceiro, a análise de energia evita desperdício: o excedente é guardado, e quando a bateria está cheia o sistema avisa quanto se perderia (por exemplo, geração de 80 kW e consumo de 30 kW com a bateria em 190 kWh: 10 kWh guardados e 40 kWh de desperdício).

6 Repositório e execução

Repositório (público): https://github.com/tvnclr/Aurora-Siger-3

Para rodar, basta `python main.py` na pasta do projeto. O `README.md` explica o funcionamento e traz exemplos de entrada e saída copiados da execução, entre eles os do próprio enunciado: energia = 40 e consumo = 70 → "ALERTA: reduzir consumo"; vento = 11 → previsão de cerca de 27 kW; geração = 40 e consumo = 70 → "ALERTA: consumo maior que geração"; geração = 80 e consumo = 30 → "SUGESTÃO: armazenar energia excedente".

Referências

Capítulos do material on-line da Fase 3 de Ciência da Computação da FIAP (2026).

Capítulo 1: A Aurora estabelece os primeiros sistemas da colônia.

Capítulo 2: A estrutura de armazenamento organizando os dados que sustentam a colônia.

Capítulo 3: A lógica estruturada que dá forma às primeiras soluções da base.

Capítulo 4: As estruturas avançadas que rastreiam recursos e informações da colônia.

Capítulo 7: A regressão inicial que permite prever variáveis da nova base.

Capítulo 8: As fontes renováveis que alimentam os primeiros sistemas da colônia.